

Komputasi Paralel - Tugas Load Balancing

Dynamic Distribution

NRP: 152024127 (Ganjil / Uneven)

Materi: Dynamic Distribution untuk Load Balancing

1. Source Code (dynamic_distribution.py)

Kode ini menggunakan multiprocessing.Pool dengan `imap_unordered` di Python. Karena waktu yang dibutuhkan tiap task berbeda-beda (uneven), pekerja (worker) yang selesai lebih cepat akan mengambil task baru dari antrian secara dinamis. Ini adalah contoh dari Dynamic Load Balancing / Distribution.

```

1 import multiprocessing
2 import time
3 import random
4
5 # untuk mensimulasikan task komputasi dengan waktu eksekusi yang bervariasi
6 def worker_task(task_id):
7     # untuk mensimulasikan beban kerja yang tidak merata (uneven workload)
8     sleep_time = random.uniform(0.1, 0.5)
9     print(f"Worker processing Task {task_id} (Expected time: {sleep_time:.2f}s)")
10    time.sleep(sleep_time)
11    return task_id, sleep_time
12
13 def dynamic_distribution():
14     num_tasks = 20
15     tasks = list(range(num_tasks))
16     num_workers = 4
17
18     print(f"NRP: 152024127 (Uneven -> Using Dynamic Distribution)")
19     print(f"Distributing {num_tasks} tasks dynamically among {num_workers} workers...")
20     print("-" * 50)
21
22     start_time = time.time()
23
24     # untuk membuat pool worker dan mendistribusikan task secara dinamis
25     # imap_unordered akan memberikan task baru ke worker yang sudah selesai (idle)
26     with multiprocessing.Pool(processes=num_workers) as pool:
27         results = pool.imap_unordered(worker_task, tasks)
28
29         total_work_time = 0
30         for task_id, duration in results:
31             total_work_time += duration
32
33     end_time = time.time()
34     actual_time = end_time - start_time
35
36     # untuk menghitung waktu optimal (total kerja dibagi jumlah worker)
37     expected_optimal = total_work_time / num_workers
38
39     print("-" * 50)
40     print("Execution Summary:")
41     print(f"Total pure work time (if sequential): {total_work_time:.4f} seconds")
42     print(f"Expected optimal time (perfect distribution): {expected_optimal:.4f} seconds")
43     print(f"Actual execution time (Dynamic Distribution): {actual_time:.4f} seconds")
44
45     # untuk menunjukkan bahwa kode mencapai waktu mendekati optimal
46     efficiency = (expected_optimal / actual_time) * 100
47     print(f"Efficiency: {efficiency:.2f}% (Shows when the code expects optimal time)")
48
49 if __name__ == '__main__':
50     dynamic_distribution()
51

```

2. Capture (Compile & Execute Output)

Berikut adalah hasil eksekusi program di terminal:

```
● kizzu@kizzu-20y10011us:~/Kuliah/Komputasi paralel/Tugas_3$ /bin/python3.12 "/home/kizzu/Kuliah/Komputasi pa
rarel/Tugas_3/dynamic_distribution.py"
NRP: 152024127 (Uneven -> Using Dynamic Distribution)
Distributing 20 tasks dynamically among 4 workers...
-----
Worker processing Task 0 (Expected time: 0.34s)
Worker processing Task 1 (Expected time: 0.49s)
Worker processing Task 2 (Expected time: 0.20s)
Worker processing Task 3 (Expected time: 0.31s)
Worker processing Task 4 (Expected time: 0.47s)
Worker processing Task 5 (Expected time: 0.26s)
Worker processing Task 6 (Expected time: 0.32s)
Worker processing Task 7 (Expected time: 0.32s)
Worker processing Task 8 (Expected time: 0.18s)
Worker processing Task 9 (Expected time: 0.46s)
Worker processing Task 10 (Expected time: 0.35s)
Worker processing Task 11 (Expected time: 0.43s)
Worker processing Task 12 (Expected time: 0.29s)
Worker processing Task 13 (Expected time: 0.25s)
Worker processing Task 14 (Expected time: 0.13s)
Worker processing Task 15 (Expected time: 0.31s)
Worker processing Task 16 (Expected time: 0.44s)
Worker processing Task 17 (Expected time: 0.35s)
Worker processing Task 18 (Expected time: 0.11s)
Worker processing Task 19 (Expected time: 0.37s)
-----
Execution Summary:
Total pure work time (if sequential): 6.3684 seconds
Expected optimal time (perfect distribution): 1.5921 seconds
Actual execution time (Dynamic Distribution): 1.7659 seconds
```

3. Penjelasan Eksekusi Task

- **Worker Pool Creation (multiprocessing.Pool):** Membuat 4 proses pekerja independen yang berjalan secara paralel. Setiap proses berjalan di core CPU yang terpisah.
- **Uneven Workload Simulation:** Tiap task diberikan beban eksekusi acak antara 0.1s hingga 0.5s. Ini mewakili workload dinamis yang tidak dapat diprediksi di dunia nyata, seperti query database atau HTTP request.
- **Dynamic Assignment (imap_unordered):** Metode ini tidak mendistribusikan task secara kaku dari awal (tidak seperti static distribution). Jika worker A kebetulan menangani task yang ringan (misal 0.11s), ia akan langsung idle dan meminta task baru berikutnya dari antrian pool. Worker yang lambat tetap memproses task lamanya tanpa menghalangi worker lain.
- **Expected Optimal Time:** Dihitung dari menjumlahkan semua pure work time secara sekuensial lalu membaginya dengan jumlah worker. Pada program ditunjukkan perbandingan Actual execution time dengan Expected optimal time, membuktikan bahwa distribusi dinamis mencapai efisiensi yang sangat tinggi (~84-90%) pada beban kerja heterogen.